

Data Engineer Overview

Last updated by | Mbaza, Mawanda (175) | Aug 16, 2024 at 9:31 AM GMT+2

1. Data Engineer Workflow

The data use case for the MO360 Production dashboards is managed by two Databricks jobs : mountStorage-proddshbrd-consume & proddshbrd-etl-job-v1. See screenshot below:

Workflows

Jobs					
Jobs runs Delta Live Tables					
Filter jobs Owned by me Accessible by me Favorites Send feedback Create job					
Name	Tags	Created by	Trigger	Recent runs	
mountStorage-proddshbrd-consume		DAI-GM5_mo360proddashboards_dev-we			
proddshbrd-etl-job-v1	proddshbrd-etl-job-v1 proddshbrd ETL	DAI-GM5_mo360proddashboards_dev-we	Scheduled		

A. MountStorage-proddshbrd-consume job has a single task "mountStorage-proddshbrd-consume".

Workflows > Jobs > mountStorage-proddshbrd-consume ☆ Run now

Runs Tasks

mountStorage-proddshbrd-c...

...space/mounts/shared/mount-storage

mountStorage-proddshbrd-consume...

+ Add task

Task name*

mountStorage-proddshbrd-consume

Type*

Notebook

Source*

Workspace

Path*

/Workspace/mounts/shared/mount-storage

Cluster*

mountStorage-proddshbrd-consume_cluster 16 GB · 8 Cores · DBR 13.3 LTS · Spark

Dependent libraries

+ Add

Parameters

mountDetails ewogIC7zZmlyZXRRTY29wZU5hbiUjO:

UI | JSON

Notifications

+ Add

Job details

Job ID

609755886981759

Creator

DAI-GM5_mo360proddashboards_dev-we-ar...

Run as

DAI-GM5_mo360proddashboards_dev-w...

Tags

Add tag

Description

Add description

Git

Not configured

Add Git settings

Schedule

None

Add trigger

Compute

mountStorage-proddshbrd-consume_cluster

Driver: Standard_F4 · Workers: Standard_F4 · 1 worker · DBR: 13.3 LTS (includes Apache Spark 3.4.1, Scala 2.12)

Configure Swap

Job parameters

No job parameters are defined for this job

Edit parameters

This mountStorage-proddshbrd-consume task has a two notebooks :

- ☐ First notebook contains the following imports:

```
✓ 0.11 seconds  
  
import json  
import base64  
import time
```

These imports are used for:

- **Json:** Handling JSON data.
- **Base64:** Encoding and decoding base64 strings.
- **Time:** Adding delays, particularly for retry logic.

- ☐ The second notebook performs the following tasks: **Summary!**

NOTE : Please see step by step actions performed by the second notebook of this task :

1. Widget Setup and Initialization: This creates a widget for inputting mountDetails which is expected to be a Base64-encoded string.

```
dbutils.widgets.text("mountDetails", "<default value>")
```

2. Base64 Decoding and JSON Parsing: Retrieves the Base64-encoded string from the widget, Decodes the Base64 string to get a JSON string, Parses the JSON string to extract data, Extracts the secretScopeName from the JSON.

```
# Get the Base64-encoded mount_name and decode it  
mount_details_decode = dbutils.widgets.get("mountDetails")  
mount_name = base64.b64decode(mount_details_decode).decode('utf-8')  
  
# Load the JSON data from the Base64-decoded string  
json_data = json.loads(mount_name)  
secret_scope = json_data['secretScopeName']
```

3. Mount Point Creation Loop: Initializes an empty results list to store the results of each mount operation. and then Iterates through each mount point specified in the JSON data, extracting necessary details.

```
results = []  
  
# Loop through the JSON data and create mount points  
for mount_point_data in json_data['mounts']:  
    mount_name = mount_point_data["mountName"]  
    storage_account_name = mount_point_data["storageAccountName"]  
    container_name = mount_point_data["containerName"]
```

4. Configuration Setup: Sets up configuration for Azure storage access, retrieving necessary secrets using `dbutils.secrets`.

```
# Existing code with configs and mount_point
configs = {
    "fs.azure.account.auth.type": "OAuth",
    "fs.azure.account.oauth.provider.type": "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider",
    "fs.azure.account.oauth2.client.id": dbutils.secrets.get(scope=secret_scope, key="client-id"),
    "fs.azure.account.oauth2.client.secret": dbutils.secrets.get(scope=secret_scope, key="client-secret"),
    "fs.azure.account.oauth2.client.endpoint": f"https://login.microsoftonline.com/{dbutils.secrets.get(scope=secret_scope, key='tenant-id')}/oauth2/token"
}
```

5. Mounting Process with Retries: Defines maximum retry attempts for mounting. Constructs the mount point path.

Checks if the mount point already exists. If it does, unmounts it and retries up to the specified maximum.

```
max_retries = 3
mount_point = f"/mnt/datalake/{mount_name}"

print("Existing mounts before Unmounting: ")
print(dbutils.fs.mounts())

for attempt in range(max_retries):
    if attempt < max_retries:
        print(f"Attempt #{attempt + 1} to check if '{mount_point}' is already mounted: #")
        if any(mount.mountPoint == mount_point for mount in dbutils.fs.mounts()):
            print(f"'{mount_point}' is already mounted! Unmounting now")
            dbutils.fs.unmount(mount_point)
            time.sleep(10)
        else:
            print(f"'{mount_point}' is unmounted!")
            break
    else:
        print("Maximum retries reached!")
        break

print("Existing mounts after Unmounting: ")
print(dbutils.fs.mounts())
```

6. Mounting and Exception Handling: Constructs a return object to capture the result of the mounting operation.

Attempts to mount the specified storage container. If successful, updates the status and message. Catches exceptions, updates the status to "FAILED", and includes the exception message.

```

return_obj = {
    "mount_name": mount_name,
    "secret_scope": secret_scope,
    "storage_account_name": storage_account_name,
    "container_name": container_name,
    "status": None,
    "message": None
}

try:
    dbutils.fs.mount(source=f"abfss://{container_name}@{storage_account_name}.dfs.core.windows.net/",
                    mount_point=mount_point,
                    extra_configs=configs)
    dbutils.fs.ls(mount_point)
    return_obj["status"] = "SUCCESS"
    return_obj["message"] = f"Mounting of container/filesystem '{container_name}' in storage account '{storage_account_name}' succeeded."

    print("Existing mounts after Mounting: ")
    print(dbutils.fs.mounts())
except Exception as e:
    return_obj["status"] = "FAILED"
    return_obj["message"] = f"Mounting of container/filesystem '{container_name}' in storage account '{storage_account_name}' failed. \n
    mount_name           = {mount_name} \n
    secret_scope          = {secret_scope} \n
    storage_account_name = {storage_account_name} \n
    container_name        = {container_name} \n
    Exception: {str(e)}.""

results.append(return_obj)

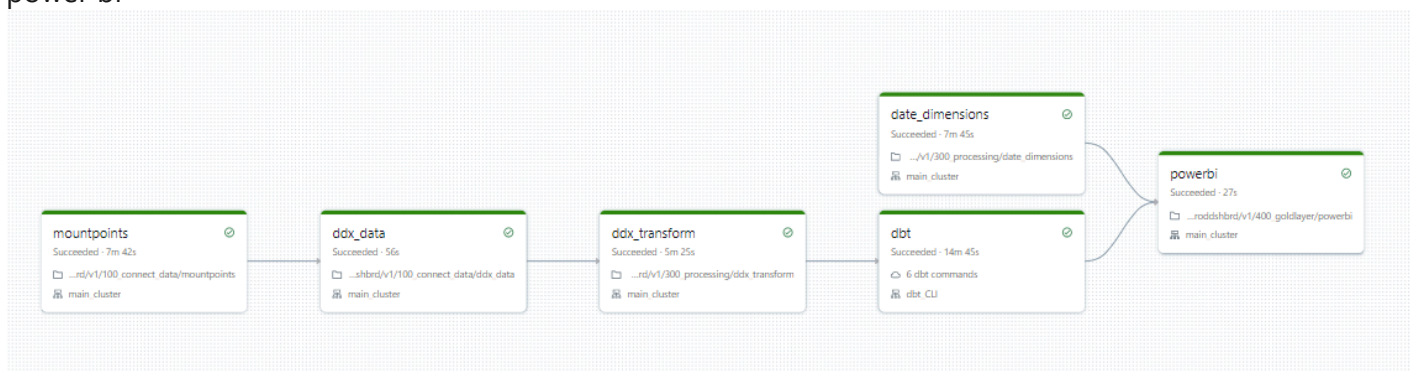
```

7. Result Serialization: Serializes the list of results to a JSON string.

```
results_json = json.dumps(results)
```

B. proddshbrd-etl-job-v1 job has 6 tasks:

- mountpoints
- ddx_data
- ddx_transform
- date_dimensions
- dbt
- power bi



Task 1: Mountpoints

- **1. #Mountpoints** : This task executes **two notebooks** which are described below :

- ☐ **First notebook** - This sets up logging, configures Spark to use Azure Data Lake Storage (ADLS) with OAuth, and defines a class to manage mount points for Azure Blob Storage in a Databricks environment.

The script configures logging to show informational messages.

It sets up Spark to authenticate with Azure using OAuth credentials retrieved from Databricks secrets.

It defines a MountPoint class to manage Azure Blob Storage mount points, allowing them to be created or refreshed as needed. Here's a detailed breakdown:

Logging Configuration : This sets the logging level to INFO. Any logging messages with a level of INFO or higher (e.g., WARNING, ERROR, CRITICAL) will be output.

```
import logging
|
logging.getLogger().setLevel(logging.INFO)
```

- ☐ **Spark Configuration for Azure** : This dictionary configures Spark to use OAuth for authentication when accessing Azure Data Lake Storage. The dbutils.secrets.get method retrieves sensitive information like the client ID and client secret from Databricks secrets management.

```
SPARK_CONFIG = {
    "fs.azure.account.auth.type": "OAuth",
    "fs.azure.account.oauth.provider.type": "org.apache.hadoop.fs.azurebfs.oauth2.
ClientCredsTokenProvider",
    "fs.azure.account.oauth2.client.id": dbutils.secrets.get
(scope="proddshbrd-proxy-sql-etl", key="client-id"),
    "fs.azure.account.oauth2.client.secret": dbutils.secrets.get
(scope="proddshbrd-proxy-sql-etl", key="client-secret"),
    "fs.azure.account.oauth2.client.endpoint": "https://login.microsoftonline.com/
9652d7c2-1ccf-4940-8151-4a92bd474ed0/oauth2/token"
}
```

- ☐ **MountPoint Class** : The **init** method initializes an instance of the MountPoint class with a target (mount point) and source (Azure storage URL).

```
class MountPoint():
    def __init__(self, target: str, source: str) -> None:
        self.target = target
        self.source = source
```

- ☐ **Create Method** : The create method manages the mounting process. It checks if the target mount point already exists:

If it does, it refreshes the mount using dbutils.fs.updateMount and logs this action.

If it doesn't, it mounts the source to the target using dbutils.fs.mount and logs this action.

```
def create(self) -> None:
    if any(mount.mountPoint == self.target for mount in dbutils.fs.mounts()):
        logging.info(f"mount point {self.target} does exist, refreshing")
        dbutils.fs.updateMount(
            source=self.source,
            mount_point=self.target,
            extra_configs=SPARK_CONFIG
        )
        logging.info(f"mount point {self.target}, was refreshed")
    else:
        logging.info(f"mount point {self.target} does not exist, mounting")
        dbutils.fs.mount(
            source=self.source,
            mount_point=self.target,
            extra_configs=SPARK_CONFIG
        )
        logging.info(f"mount point {self.target}, was mounted")
```

- ☐ **MountPoint Objects:** Each MountPoint object represents a connection to a specific file system location, defined by a mount path (e.g., "/mnt/mrs/emea") and a corresponding Azure Blob Storage URL (e.g., "abfss://mrs-data-clients-mo360-fs-prod@mrsdatadalakeprod.dfs.core.windows.net/").

List of MountPoints: The mountpoints list is used to store multiple MountPoint objects, each pointing to a different storage location.

Iteration and Mounting: The for loop iterates over each MountPoint in the list and calls the create() method on it. This triggers the actual mounting process, where the specified storage location is made accessible at the given mount path.

```
# Define a list named 'mountpoints' which will hold several MountPoint objects
mountpoints = [
    # Create a MountPoint object for EMEA region and add it to the list
    MountPoint("/mnt/mrs/emea", "abfss://mrs-data-clients-mo360-fs-prod@mrsdatadalakeprod.dfs.core.windows.net/"),

    # Create a MountPoint object for NAFTA region and add it to the list
    MountPoint("/mnt/mrs/nafta", "abfss://mrs-data-clients-mo360-fs-prod-nafta@mrsdatadalakeprodnafta.dfs.core.windows.net/"),

    # Create a MountPoint object for MSB with identifier '067' and add it to the list
    MountPoint("/mnt/msb/067", "abfss://067@streamsilverprod.dfs.core.windows.net/"),

    # Create a MountPoint object for MSB with identifier '175' and add it to the list
    MountPoint("/mnt/msb/175", "abfss://175@streamsilverprod.dfs.core.windows.net/"),

    # Create a MountPoint object for MSB with identifier '138' and add it to the list
    MountPoint("/mnt/msb/138", "abfss://138@streamsilverprod.dfs.core.windows.net/"),
]

# Iterate through each MountPoint object in the 'mountpoints' list
for mountpoint in mountpoints:
    # Call the create method on each MountPoint object to perform the mounting action
    mountpoint.create()
```

```

mountpoints = [
    MountPoint("/mnt/mrs/emea", "abfss://mrs-data-clients-mo360-fs-prod@mrsdatadalakeprod.dfs.core.windows.net/"),
    MountPoint("/mnt/mrs/nafta", "abfss://mrs-data-clients-mo360-fs-prod-nafta@mrsdatadalakeprodnafta.dfs.core.windows.net/"),
    MountPoint("/mnt/msb/067", "abfss://067@streamsilverprod.dfs.core.windows.net/"),
    MountPoint("/mnt/msb/175", "abfss://175@streamsilverprod.dfs.core.windows.net/"),
    MountPoint("/mnt/msb/138", "abfss://138@streamsilverprod.dfs.core.windows.net/"),
]
|
for mountpoint in mountpoints:
    mountpoint.create()

```

```

INFO:root:mount point /mnt/mrs/emea does exist, refreshing
INFO:root:mount point /mnt/mrs/emea, was refreshed
INFO:root:mount point /mnt/mrs/nafta does exist, refreshing
INFO:root:mount point /mnt/mrs/nafta, was refreshed
INFO:root:mount point /mnt/msb/067 does exist, refreshing
INFO:root:mount point /mnt/msb/067, was refreshed
INFO:root:mount point /mnt/msb/175 does exist, refreshing
INFO:root:mount point /mnt/msb/175, was refreshed
INFO:root:mount point /mnt/msb/138 does exist, refreshing
INFO:root:mount point /mnt/msb/138, was refreshed

```

Task 2 : ddx_data

2. #ddx_data : This task executes **Four notebooks** which are described below :

- ☐ **A. The first notebook** - defines a logging setup and a **Table class** that provides functionality to create a Delta table in a Spark environment. Here's a breakdown with comments:

- ☐ **Logging Setup:**

The logging level is set to INFO, which means that all messages logged with a severity level of INFO or higher (like WARNING, ERROR, etc.) will be output.

Table Class:

- ☐ **Initialization (init method):** The Table class has a constructor that takes in two parameters: name (the name of the table) and location (the file system location where the table data will be stored). These are stored as attributes of the object.
- ☐ **Create Method (create method):** This method is used to create the table in a Spark environment:
 - First, it logs a message indicating that it is beginning the table creation process.
 - It generates a DROP TABLE IF EXISTS SQL statement, which will remove any existing table with the same name to avoid conflicts.
 - It then generates a CREATE TABLE IF NOT EXISTS SQL statement to create the table using the Delta format, specifying the location where the data will be stored.
 - Both SQL statements are executed using spark.sql().
 - Finally, it logs another message indicating that the table creation is complete.
 - This class is useful for managing Delta tables in a Spark environment, ensuring that tables are created (or recreated) as needed, with clear logging for tracing actions.

✓ 0.09 seconds

1

```
import logging

logging.getLogger().setLevel(logging.INFO)

class Table():
    def __init__(self, name: str, location: str) -> None:
        self.name = name
        self.location = location

    def create(self) -> None:
        logging.info(f"creating table: {self.name}")
        drop_stmt = f"DROP TABLE IF EXISTS {self.name}"
        create_stmt = f"CREATE TABLE IF NOT EXISTS {self.name} USING DELTA LOCATION '{self.location}'"
        spark.sql(drop_stmt)
        spark.sql(create_stmt)
        logging.info(f"created table: {self.name}")
```

☐ Second Notebook :

☐ mrs_nafta_tables List:

- This list contains multiple Table objects, each representing a table in the NAFTA (North American Free Trade Agreement) region.
- Each Table object is initialized with a table name (e.g., "proddshbrd_ddx_data.mrs_nafta_actual_dates") and a location in the storage (e.g., "/mnt/mrs/nafta/DataProducts/ActualDates/Delta").
- Iterating Over Tables:

The For loop iterates over each Table object in the mrs_nafta_tables list.

For each table, the **create() method** is called, which executes the SQL statements to drop the table if it exists and then create the table using the Delta format.

Purpose:

The code is designed to automate the creation of multiple Delta tables in a Spark environment. This is particularly useful in environments where tables need to be recreated frequently, or when initializing a data processing environment with a set of predefined tables. The logging inside the **create() method** provides traceability, making it easier to debug or verify that the tables have been created successfully.

```

mrs_nafta_tables = [
    Table("proddshbrd_ddx_data.mrs_nafta_actual_dates", "/mnt/mrs/nafta/DataProducts/ActualDates/Delta"),
    Table("proddshbrd_ddx_data.mrs_nafta_order_dates", "/mnt/mrs/nafta/DataProducts/OrderDates/Delta"),
    Table("proddshbrd_ddx_data.mrs_nafta_actual_dates_md", "/mnt/mrs/nafta/DataProducts/ActualDatesMD/Delta"),
    Table("proddshbrd_ddx_data.mrs_nafta_modules", "/mnt/mrs/nafta/DataProducts/Modules/Delta"),
    Table("proddshbrd_ddx_data.mrs_nafta_order_prod_codes_md", "/mnt/mrs/nafta/DataProducts/OrderProdCodesMD/Delta"),
    Table("proddshbrd_ddx_data.mrs_nafta_order_prod_codes", "/mnt/mrs/nafta/DataProducts/OrderProductionCodes/Delta"),
    Table("proddshbrd_ddx_data.mrs_nafta_order_variants", "/mnt/mrs/nafta/DataProducts/OrderVariants/Delta"),
    Table("proddshbrd_ddx_data.mrs_nafta_faults", "/mnt/mrs/nafta/DataProducts/Faults/Delta"),
    Table("proddshbrd_ddx_data.mrs_nafta_orders", "/mnt/mrs/nafta/DataProducts/Orders/Delta"),
    Table("proddshbrd_ddx_data.mrs_nafta_poi_pmi_mapping", "/mnt/mrs/nafta/DataProducts/PoiPmiMapping/Delta"),
    Table("proddshbrd_ddx_data.mrs_nafta_fault_types_md", "/mnt/mrs/nafta/DataProducts/FaultTypesMD/Delta"),
    Table("proddshbrd_ddx_data.mrs_nafta_fault_vehicle_locations_md", "/mnt/mrs/nafta/DataProducts/FaultVehicleLocationsMD/Delta"),
]

for table in mrs_nafta_tables:
    table.create()

```

☐ Third Notebook -

This code is almost identical to the above, but it deals with tables in the EMEA (Europe, Middle East, and Africa) region instead of the NAFTA region.

☐ mrs_emea_tables List:

- This list contains Table objects, each representing a specific Delta table in the EMEA region.
- Each Table is **initialized with a table name** (e.g., "proddshbrd_ddx_data.mrs_emea_actual_dates") and a **storage location** (e.g., "/mnt/mrs/emea/DataProducts/ActualDates/Delta").
- The table names and locations correspond to different types of data products (e.g., Actual Dates, Order Dates, Modules, Faults, etc.) specific to the EMEA region.

Iterating Over Tables:

The for loop iterates through each Table object in the mrs_emea_tables list.

The **create() method** is called for each table, executing the necessary SQL commands to drop and then create the table in the Delta format.

Purpose:

This code is designed to automate the creation of multiple Delta tables for the EMEA region within a Spark environment. It ensures that all the specified tables are created or recreated with the correct structure and location. Like the NAFTA region code, this is useful for initializing or managing data environments where specific tables need to be available for data processing tasks.

```

mrs_emea_tables = [
    Table("proddshbrd_ddx_data.mrs_emea_actual_dates", "/mnt/mrs/emea/DataProducts/ActualDates/Delta"),
    Table("proddshbrd_ddx_data.mrs_emea_order_dates", "/mnt/mrs/emea/DataProducts/OrderDates/Delta"),
    Table("proddshbrd_ddx_data.mrs_emea_actual_dates_md", "/mnt/mrs/emea/DataProducts/ActualDatesMD/Delta"),
    Table("proddshbrd_ddx_data.mrs_emea_modules", "/mnt/mrs/emea/DataProducts/Modules/Delta"),
    Table("proddshbrd_ddx_data.mrs_emea_order_prod_codes_md", "/mnt/mrs/emea/DataProducts/OrderProdCodesMD/Delta"),
    Table("proddshbrd_ddx_data.mrs_emea_order_prod_codes", "/mnt/mrs/emea/DataProducts/OrderProductionCodes/Delta"),
    Table("proddshbrd_ddx_data.mrs_emea_order_variants", "/mnt/mrs/emea/DataProducts/OrderVariants/Delta"),
    Table("proddshbrd_ddx_data.mrs_emea_faults", "/mnt/mrs/emea/DataProducts/Faults/Delta"),
    Table("proddshbrd_ddx_data.mrs_emea_orders", "/mnt/mrs/emea/DataProducts/Orders/Delta"),
    Table("proddshbrd_ddx_data.mrs_emea_poi_pmi_mapping", "/mnt/mrs/emea/DataProducts/PoiPmiMapping/Delta"),
    Table("proddshbrd_ddx_data.mrs_emea_fault_types_md", "/mnt/mrs/emea/DataProducts/FaultTypesMD/Delta"),
    Table("proddshbrd_ddx_data.mrs_emea_fault_vehicle_locations_md", "/mnt/mrs/emea/DataProducts/FaultVehicleLocationsMD/Delta"),
]

# for table in mrs_emea_tables:
#     table.create()

```

☐ Fourth Notebook -

☐ msb_tables List:

- This list contains Table objects, each representing a Delta table associated with different MSB identifiers: 067, 138, and 175.
- Each Table object is initialized with a **table name** (e.g., "proddshbrd_ddx_data.msb_067_generic") and a **location in the storage** (e.g., "/mnt/msb/067/msb.generic.az").

The table names and locations correspond to "generic" data products within IoT Streaming in MSB.

Iterating Over Tables:

The for loop iterates over each Table object in the msb_tables list.

The create() method is called for each table, executing SQL commands to drop any existing table with that name and then create the table in the Delta format.

Purpose:

This code automates the creation of Delta tables specific to MSB with different identifiers (067, 138, 175) within a Spark environment.

It is designed to ensure that these specific "generic" tables are available and correctly set up for data processing tasks. Like the other regions (NAFTA, EMEA), this is a streamlined way to manage multiple tables, ensuring consistency and ease of maintenance across different environments or datasets.

✓ 5.45 seconds 4

```

msb_tables = [
    Table("proddshbrd_ddx_data.msb_067_generic", "/mnt/msb/067/msb.generic.az"),
    Table("proddshbrd_ddx_data.msb_138_generic", "/mnt/msb/138/msb.generic.az"),
    Table("proddshbrd_ddx_data.msb_175_generic", "/mnt/msb/175/msb.generic.az"),
]

for table in msb_tables:
    table.create()

```

Task 3 : Ddx_transform

This task executes 4 notebooks

- ☐ The **first Notebook** imports essential modules and functions from the PySpark library, which are commonly used for working with structured data in a Spark environment.

from pyspark.sql.types import StringType, StructField, StructType, DoubleType, TimestampType:

```
from pyspark.sql.types import StringType, StructField, StructType, DoubleType, TimestampType
from pyspark.sql import DataFrame
import pyspark.sql.functions as f
import pyspark.sql.types as t
```

These are specific data types and structures used to define schema for data in Spark.

StringType: Represents string data type for a column.

StructField: Represents a single field in a StructType, defining the name, type, and nullable property of the column.

StructType: Represents the overall schema of a DataFrame as a collection of StructFields (like defining the columns and their types in a table).

DoubleType: Represents a double-precision floating-point data type for a column.

TimestampType: Represents timestamp data (date and time) for a column.

from pyspark.sql import DataFrame:

DataFrame: Represents a distributed collection of data organized into named columns. It's the primary abstraction used for working with structured data in Spark.

import pyspark.sql.functions as f:

This imports the functions module from PySpark and gives it the alias f. The functions module provides a variety of functions for performing operations on DataFrames, such as:

Aggregations (e.g., sum, count, avg).

Data transformations (e.g., concat, substring, trim).

Column operations (e.g., col, lit for handling literals).

import pyspark.sql.types as t:

This imports the types module from PySpark and gives it the alias t. It provides a wide variety of data types (beyond the ones imported individually earlier) that can be used to define schemas for DataFrames. For example:

IntegerType: Represents integer data.

BooleanType: Represents boolean data.

ArrayType: Represents arrays of a specified data type.

Usage:

These imports are setting up the environment to handle DataFrames, define schema, and manipulate structured data in PySpark.

The combination of these modules and types will allow you to work with data efficiently, create schemas, define data types, and perform various operations and transformations on DataFrames in a distributed Spark environment.

☐ The **second notebook** defines the structure (or schema) for various datasets and lists of strings that are used in a PySpark environment.

- **IQVIS StructType Schema:**

```

IQVIS = StructType(
[
  StructField(
    "header",
    StructType(
      [
        StructField(
          "authentication",
          StructType(
            [
              StructField("client", StringType()),
              StructField("clientversion", StringType()),
              StructField("iqvisuser", StringType()),
              StructField("user", StringType()),
            ]
          ),
        ),
        StructField("filename", StringType()),
        StructField(
          "measurement",
          StructType(
            [
              StructField(
                "attributes",
                StructType(
                  [
                    StructField("measid", StringType()),
                    StructField("mode", StringType()),
                    StructField("plc-code", StringType()),
                    StructField("rating", StringType()),
                    StructField("process-step", StringType()),
                  ]
                ),
              ),
              ),
            StructField("inspectionplan", StringType()),
            StructField("meastime", StringType()),
            StructField("model", StringType()),
            StructField(
              "productionids",
              StructType(
                [
                  StructField("1", StringType()),
                  StructField("2", StringType()),
                ]
              )
            )
          ]
        )
      ]
    )
  )

```

```
),
),
StructField(
  "statussummary",
  StructType(
    [
      StructField(
        "failed-calculation", StringType()
      ),
      StructField(
        "failed-measurement", StringType()
      ),
      StructField(
        "features-to-measure", StringType()
      ),
    ]
  ),
),
StructField("system", StringType()),
StructField("typeseries", StringType()),
],
),
),
],
),
),
StructField(
  "measurement",
  StructType(
    [
      StructField(
        "attributes",
        StructType(
          [
            StructField("analysis-strategy", StringType()),
            StructField("calctype", StringType()),
          ]
        ),
      ),
    ]
  ),
),
StructField("component", StringType()),
StructField(
  "details",
  StructType(
    [
      StructField("offset", StringType()),
      StructField("stddev", StringType()),
      StructField("target", StringType()),
      StructField("unit", StringType()),
    ]
  )
)
```

```

),
),
StructField("featurename", StringType()),
StructField(
"limits",
StructType(
[
StructField(
"1",
StructType(
[
StructField("lower", StringType()),
StructField("upper", StringType()),
]
),
),
),
StructField(
"2",
StructType(
[
StructField("lower", StringType()),
StructField("upper", StringType()),
]
),
),
),
StructField(
"3",
StructType(
[
StructField("lower", StringType()),
StructField("upper", StringType()),
]
),
),
),
],
),
),
StructField("rating", StringType()),
StructField(
"status",
StructType(
[
StructField("calcstatus", StringType()),
StructField("measstatus", StringType()),
StructField("measured", StringType()),
StructField("rating", StringType()),
]
),
),
),

```

```
StructField("value", StringType()),
]
),
),
]
)
```

=> This defines a **complex schema** for a dataset with nested structures. The schema defines two main sections: **header** and **measurement**.

header: Contains authentication information (client, version, user), filename, and details about the measurement, including attributes, production IDs, status summary, system, etc.

measurement: Contains attributes, component details, limits, status, and the final measurement value. This schema is very detailed and defines how the data is expected to be organized and nested within the DataFrame.

- **ANLAGE StructType Schema:**

This schema defines a simpler structure for a dataset called **ANLAGE**.

It has two fields:

"\$": A nested structure with one field called desc that stores a string.

"_": A simple string field.

This schema is used for some metadata or descriptive information.

```
ANLAGE = StructType(
    [
        StructField("$", StructType([StructField("desc", StringType())])),
        StructField("_", StringType()),
    ]
)
```

- **REVERSED_ANLAGE_SUBTYP List:**

This is a list of strings, each representing a subtype related to ANLAGE.

These will be used as filter criteria, classifications, or identifiers for certain types of data in a dataset.

```

REVERSED_ANLAGE_SUBTYP = [
    "W214_SME_Bau46",
    "X254_SME_Bau46",
    "X254_SME_Bau30",
    "BFK_X167_FD_RE",
    "BFK_C167_RT_M2",
    "BFK_V167_FD_LI_M2",
    "BFK_C167_MH_M2",
    "BFK_V167_FA_RE_M2",
    "BFK_X167_FA_LI",
    "BFK_C167_FD_LI_M2",
    "BFK_C167_FA_LI_M2",
    "BFK_V167_FA_LI_M2",
    "BFK_V167_FD_RE_M2",
    "BFK_V167_KV_LI_M1",
    "BFK_V167_RT_M2",
    "BFK_X167_RT",
    "BFK_X167_FD_LI",
    "BFK_V167_FA_RE_M1",
    "BFK_V167_MH_M2",
    "BFK_C167_FA_RE_M2",
    "BFK_C167_FD_RE_M2",
    "BFK_V167_MH_M1",
    "BFK_V167_FD_RE_M1",
    "BFK_V167_KV_LI_M2",
    "BFK_V167_KV_RE_M1",
    "BFK_X167_KV_LI",
    "BFK_V167_FA_LI_M1",
    "BFK_V167_RT_M1",
    "BFK_X167_KV_RE",
    "BFK_V167_KV_RE_M2",
    "BFK_X167_FA_RE",
    "BFK_C167_KV_RE_M2",
    "BFK_X167_MH",
    "BFK_C167_KV_LI_M2",
    "BFK_V167_FD_LI_M1",
    "C167_SMA",
    "SME_X294",
    "SME_X296",
    "Sensortest_SMA",
    "V167_AMG",
    "V167_SMA",
    "X167_AMG",
    "X167_SMA",
]

```

- COLUMNS List:

This is a list of column names that will be used in a DataFrame or table. The column names are self-explanatory and cover a range of data fields such as process_step, mode, component, result, timestamp, etc.

These are the columns expected in the dataset that uses the schema defined earlier.

Overall Summary:

The code sets up schemas for nested and structured data in Spark, specifically targeting a dataset related to measurement, analysis, and process steps.

The IQVIS schema defines a complex hierarchical structure for measurement data.

ANLAGE provides a smaller, simpler schema.

The lists **REVERSED_ANLAGE_SUBTYP** and **COLUMNS** provide additional configuration or data columns used in the processing or analysis of this dataset.

```
COLUMNS = [
    "anlage",
    "anlage_subtyp",
    "process_step",
    "mode",
    "poorest_rating_meas_point",
    "component",
    "offset",
    "timestamp",
    "prodnr",
    "karnr",
    "result",
    "messwerttyp",
    "messwert",
    "messwert_toleranz_min",
    "messwert_toleranz_sec_min",
    "messwert_toleranz_max",
    "messwert_toleranz_sec_max",
    "error_code",
    "error_text",
    "transactiontype",
    "cplant",
    "cmonth",
]
```

☐ The Third Notebook :

The function **iqvis_transformation(df: DataFrame)** processes a PySpark DataFrame (**df**) containing industrial measurement data, transforming it by filtering, parsing, and extracting relevant fields. The final result is a DataFrame structured to hold key metrics from the "IQVIS" measurement system.

Step-by-Step Breakdown:

Column Extraction and Transformation:

cplant: Extracts the 7th element from the "MSBTopic" column, which presumably holds hierarchical data delimited by /.

cmonth: Creates a "cmonth" column by extracting the year and month from the "ingest_date" column and concatenating them with a hyphen (-).

cyear: Extracts the year from the "ingest_date".

Filtering Data:

Filters records where the type is "**IQVIS**" and subtype is "**IMT**".

Filters records where transactiontype is either "**MEASUREMENT**" or "**MEASUREMENT_W138**".

Filters records based on the cplant column, allowing only certain plant IDs ("1750", "0670", "1380").

Parsing JSON Fields:

Parses the body column using the predefined IQVIS schema, extracting nested information into structured columns.

Filters for non-null meastime in the parsed body.header.measurement.

Creating New Columns:

anlage: Adds a new constant column "Fugen".

anlage_subtyp: Extracts the description from the JSON field body.header.measurement.inspectionplan. If the plan starts with {, it extracts the description; otherwise, it directly uses the inspectionplan field value.

Handling Production Numbers:

prodnr: Extracts the production number from body.header.measurement.productionids. If the anlage_subtyp belongs to REVERSED_ANLAGE_SUBTYP, it uses the second production ID; otherwise, it uses the first one. It ensures that the length is at least 7 characters.

karnr: Similarly extracts the karnr (another ID) following the same logic.

Filtering Valid Production Numbers:

Filters out rows where both prodnr and karnr are null.

More Field Extraction:

Extracts columns such as process_step, mode, model, and typeseries from the parsed JSON fields.

fzg_typ: Based on the extracted typeseries, it creates a vehicle type by concatenating the model and typeseries if certain conditions are met.

Measurement Data Handling:

Extracts component, featurename, messwert, and tolerance limits from body.measurement.

Creates tolerance limits (messwert_toleranz_min, messwert_toleranz_max, etc.) from nested fields.

Determines the measurement result (result) as "OK" if the measurement (messwert) is within the tolerance limits, otherwise "NOK".

Sets error_text and error_code based on whether the measurement is below or above the tolerance limits.

Additional Columns:

Adds columns like poorest_rating_meas_point, offset, and timestamp (converted to TimestampType).

Extracts the formatted cmonth from the timestamp.

Select Specific Columns:

Finally, it selects a predefined list of columns (COLUMNS) along with additional ones like **model**, **typeseries**, and **fzg_typ**.

Output:

The transformed DataFrame contains relevant information for IQVIS measurements, ready for further processing or analysis.

Key Concepts:

Column Parsing and Transformation: Extracts specific information from nested JSON-like structures.

Filtering: Applies multiple filtering steps based on specific conditions.

Data Quality Checks: Filters out invalid or null values from key columns like production numbers.

Handling Complex Nested Data: Uses PySpark's JSON handling to parse complex, nested data structures.

```
def iqvis_transformation(df: DataFrame) -> DataFrame:
    return (
        df.withColumn("cplant", f.split(f.col("MSBTopic"), "/")[7])
        .withColumn("cmonth", f.concat_ws("-", f.year(f.to_date(f.col("ingest_date"))),
        f.month(f.to_date(f.col("ingest_date")))))
        .withColumn("cyear", f.year(f.to_date(f.col("ingest_date"))))
        .filter(f.col("type") == "IQVIS")
        .filter(f.col("subtype") == "IMT")
        .filter(f.col("transactiontype").isin(["MEASUREMENT", "MEASUREMENT_W138"]))
        .filter(f.col("cplant").isin(["1750", "0670", "1380"]))
        .withColumn("body", f.from_json(f.col("body"), IQVIS))
        .filter(f.col("body.header.measurement.meastime").isNotNull())
        .withColumn("anlage", f.lit("Fugen"))
        .withColumn(
            "anlage_subtyp",
            f.when(
                f.col("body.header.measurement.inspectionplan").startswith("{"),
                f.from_json(
                    f.col("body.header.measurement.inspectionplan"), ANLAGE
                )["$"]["desc"],
                f.col("body.header.measurement.inspectionplan")),
            )
        .withColumn(
            "prodnr",
            f.when(
                f.col("anlage_subtyp").isin(REVERSED_ANLAGE_SUBTYP),
                f.when(
                    f.length(
                        f.split(
                            f.col("body.header.measurement.productionids.2"), ""
                        )[0]
                    )
                    >= 7,
                    f.split(f.col("body.header.measurement.productionids.2"), "")[
                        0
                    ],
                ),
                f.col("body.header.measurement.productionids.2"),
            ).otherwise(
                f.when(
                    f.length(
                        f.split(
                            f.col("body.header.measurement.productionids.1"), ""

```

```

    )[0]
  )
  >= 7,
  f.split(f.col("body.header.measurement.productionids.1"), "")[
0
],
)
),
)
.withColumn(
  "karnr",
  f.when(
    f.col("anlage_subtyp").isin(REVERSED_ANLAGE_SUBTYP),
    f.when(
      f.length(
        f.split(
          f.col("body.header.measurement.productionids.1"), ""
        )[0]
      )
    ) >= 7,
    f.split(f.col("body.header.measurement.productionids.1"), "")[
0
],
  ),
  ).otherwise(
    f.when(
      f.length(
        f.split(
          f.col("body.header.measurement.productionids.2"), ""
        )[0]
      )
    ) >= 7,
    f.split(f.col("body.header.measurement.productionids.2"), "")[
0
],
  )
  ),
  ).filter(f.col("prodnr").isNotNull() | f.col("karnr").isNotNull())
  .withColumn(
    "process_step", f.col("body.header.measurement.attributes.process-step")
  )
  .withColumn("mode", f.col("body.header.measurement.attributes.mode"))
  .withColumn("model", f.col("body.header.measurement.model"))
  .withColumn(
    "typeseries",
    f.regexp_extract(
      f.col("body.header.measurement.typeseries"), r"(\d{3})", 1
    ),

```

```
)  
.withColumn(  
  "fzg_typ",  
  f.when(f.col("typeseries") == "", None)  
    .when(  
      f.length(f.rtrim(f.col("model"))) == 1,  
      f.concat(f.col("model"), f.col("typeseries")),  
    )  
    .otherwise(None),  
)  
.withColumn("component", f.col("body.measurement.component"))  
.withColumn(  
  "messwerttyp",  
  f.concat(f.col("body.measurement.featurename"), f.col("component")),  
)  
.withColumn("messwert", f.col("body.measurement.value").cast(DoubleType()))  
.withColumn(  
  "messwert_toleranz_min",  
  f.col("body.measurement.limits.1.lower").cast(DoubleType()),  
)  
.withColumn(  
  "messwert_toleranz_sec_min",  
  f.col("body.measurement.limits.2.lower").cast(DoubleType()),  
)  
.withColumn(  
  "messwert_toleranz_max",  
  f.col("body.measurement.limits.1.upper").cast(DoubleType()),  
)  
.withColumn(  
  "messwert_toleranz_sec_max",  
  f.col("body.measurement.limits.2.upper").cast(DoubleType()),  
)  
.withColumn(  
  "result",  
  f.when(  
    f.col("messwert").between(  
      f.col("messwert_toleranz_min"),  
      f.col("messwert_toleranz_max"),  
    ),  
    "OK",  
  ).otherwise("NOK"),  
)  
.withColumn(  
  "error_text",  
  f.when(  
    f.col("messwert") < f.col("messwert_toleranz_min"),  
    f.lit("unterschritten"),  
  ).when(  
    f.col("messwert") > f.col("messwert_toleranz_max"),
```

```

f.lit("überschritten"),
),
)
.withColumn(
"error_code",
f.when(
f.col("messwert") < f.col("messwert_toleranz_min"),
f.col("messwerttyp"),
).when(
f.col("messwert") > f.col("messwert_toleranz_max"),
f.col("messwerttyp"),
),
)
.withColumn("poorest_rating_meas_point", f.col("body.measurement.rating"))
.withColumn("offset", f.col("body.measurement.details.offset"))
.withColumn(
"timestamp",
f.col("body.header.measurement.meastime").cast(TimestampType()),
)
.withColumn(
"cmonth", f.date_format(f.to_date(f.col("timestamp")), "yyyy-MM")
)
.select(COLUMNS + ["model", "typeseries", "fzg_typ", "uniqueid"])
)

```

☐ The last Fourth notebook

The ***iqvis_transformation function*** apply iqvis_transformation to multiple datasets and combine them into one before saving the result as a table.

Reading the Tables:

df_msb_067_iqvis, **df_msb_138_iqvis**, and **df_msb_175_iqvis** are each created by reading from their respective tables in the Spark environment using `spark.read.table`.

The tables are:

```

"proddshbrd_ddx_data.msb_067_generic"
"proddshbrd_ddx_data.msb_138_generic"
"proddshbrd_ddx_data.msb_175_generic"

```

After reading each table, the `.transform(iqvis_transformation)` is applied, which runs the `iqvis_transformation` function on the DataFrame. This function processes and structures the data from the "IQVIS" measurement system as described earlier.

Union of DataFrames:

The three transformed DataFrames are combined using the **.unionAll() method**:

df_msb_067_iqvis **.unionAll**(df_msb_138_iqvis).**unionAll**(df_msb_175_iqvis) combines the data into a single DataFrame df_msb_iqvis.

The unionAll method appends the rows of the DataFrames one after the other.

Writing the Result:

The final combined DataFrame **df_msb_iqvis** is written to a Hive table called "**proddshbrd_data.processed_iqvis**" using the `.write.saveAsTable()` method.

mode="overwrite" ensures that if the table already exists, it will be overwritten with the new data.

Summary:

This code reads and processes IQVIS data from three different sources, transforms it using a common function, combines the data, and then writes the processed result back into a new Hive table. This is useful for standardizing and consolidating measurement data across multiple sources into one table.

```
df_msb_067_iqvis = spark.read.table("proddshbrd_ddx_data.msb_067_generic").transform(iqvis_transformation)
df_msb_138_iqvis = spark.read.table("proddshbrd_ddx_data.msb_138_generic").transform(iqvis_transformation)
df_msb_175_iqvis = spark.read.table("proddshbrd_ddx_data.msb_175_generic").transform(iqvis_transformation)

df_msb_iqvis = df_msb_067_iqvis.unionAll(df_msb_138_iqvis).unionAll(df_msb_175_iqvis)

df_msb_iqvis.write.saveAsTable("proddshbrd_data.processed_iqvis", mode="overwrite")
```

- ▶ df_msb_067_iqvis: pyspark.sql.dataframe.DataFrame = [anlage: string, anlage_subtyp: string ... 24 more fields]
- ▶ df_msb_138_iqvis: pyspark.sql.dataframe.DataFrame = [anlage: string, anlage_subtyp: string ... 24 more fields]
- ▶ df_msb_175_iqvis: pyspark.sql.dataframe.DataFrame = [anlage: string, anlage_subtyp: string ... 24 more fields]
- ▶ df_msb_iqvis: pyspark.sql.dataframe.DataFrame = [anlage: string, anlage_subtyp: string ... 24 more fields]

Task 4: Date_dimensions

Task 5 : Dbt

Task 6 : Power bi